Day 9 – Palindrome & Anagram

1️⃣ Palindrome String

Palindrome kya hota hai?

Aisa string jo same forward & backward ho.

Example:

"madam" → palindrome ✅
"hello" → not palindrome ❌

- Approach 1 – Reverse karke check

```
public class PalindromeString {
   public static void main(String[] args) {

       String str = "madam";
       String rev = "";

       for(int i = str.length()-1; i >= 0; i--){
           rev = rev + str.charAt(i);
       }

       if(str.equals(rev)){
           System.out.println("Palindrome");
       } else {
           System.out.println("Not Palindrome");
       }

   }
}
```

Time Complexity: $O(n^2)$ ❌ (string concatenation slow hai)

- Approach 2 – Two Pointer (BEST)

```
public class PalindromeOptimized {
    public static void main(String[] args) {

        String str = "madam";

        int left = 0;
        int right = str.length() - 1;
        boolean isPalindrome = true;

        while(left < right){

            if(str.charAt(left) != str.charAt(right)){
                isPalindrome = false;
                break;
            }

            left++;
            right--;
        }

        if(isPalindrome){
            System.out.println("Palindrome");
        } else {
            System.out.println("Not Palindrome");
        }

    }
}
```

Time Complexity: O(n) ✅
Space Complexity: O(1)

2️⃣ Anagram Problem

Anagram kya hota hai?

Do strings same characters se bane ho, order different ho sakta hai.

Example:

"listen" & "silent" → Anagram ✅
"hello" & "world" → Not Anagram ❌

- Approach 1 – Sorting (Easy & Important)

Logic:

Dono strings ko char array me convert karo

Sort karo

Compare karo

Java Code

```
package Day_9;

import java.util.Arrays;

//Apporach 1
public class AnagramCheck {
    // anagram do String same characters se bane ho, order different ho skta hai
    public static void main(String[] args) {

    String str1="Listen";
    String str2="silent";

    // length check
    if(str1.length() != str2.length()){
        System.out.println("Not Anagram");
        return;
    }

    char[] arr1= str1.toCharArray();
    char[] arr2=str2.toCharArray();

    Arrays.sort(arr1);
    Arrays.sort(arr2);

    if(Arrays.equals(arr1, arr2)){
        System.out.println("Anagram");
    }
    else{
        System.out.println("Not Anagram");
    }

    }
}

```

Time Complexity: O(n log n)

- Approach 2 – Without Sorting (Better)

```java
public class AnagramWithoutSorting {
   public static void main(String[] args) {

       String s1 = "listen";
       String s2 = "silent";

       if(s1.length() != s2.length()){
           System.out.println("Not Anagram");
           return;
       }

       int[] count = new int[26];

       for(int i = 0; i < s1.length(); i++){
           count[s1.charAt(i) - 'a']++;
           count[s2.charAt(i) - 'a']--;
       }

       for(int i = 0; i < 26; i++){
           if(count[i] != 0){
               System.out.println("Not Anagram");
               return;
           }
       }

       System.out.println("Anagram");
   }
}
```

Time Complexity: O(n) ✅
Space Complexity: O(1)

Dry Run:

- Code Part

```
for(int i = 0; i < s1.length(); i++){
   count[s1.charAt(i) - 'a']++;
   count[s2.charAt(i) - 'a']--;
}
```

- Example

s1 = "listen"
s2 = "silent"

🔹 Concept

count[26] array bana hai (a–z ke liye)

s1 ke characters → +1

s2 ke characters → -1

👉 Agar dono strings same characters rakhte hain → final array sab 0 ho jayega ✅

🔥 Step-by-Step Dry Run

Initial:

count = [0,0,0,0,...,0] // size = 26

✅ i = 0

s1[0] = 'l'
s2[0] = 's'

Index nikalte hain:

'l' - 'a' = 11
's' - 'a' = 18

Update:

count[11]++ → l = 1
count[18]-- → s = -1

✅ i = 1

s1[1] = 'i'
s2[1] = 'i'

Index:

'i' - 'a' = 8

Update:

count[8]++ → 1
count[8]-- → 0

👉 cancel ho gaya ✔️

✅ i = 2

s1[2] = 's'
s2[2] = 'l'

Index:

s → 18
l → 11

Update:

count[18]++ → (-1 + 1 = 0)
count[11]-- → (1 - 1 = 0)

👉 dono cancel ho gaye ✔️

✅ i = 3

s1[3] = 't'
s2[3] = 'e'

t → 19
e → 4

count[19]++ → 1
count[4]-- → -1

✅ i = 4

s1[4] = 'e'
s2[4] = 'n'

e → 4
n → 13

count[4]++ → (-1 + 1 = 0)
count[13]-- → -1

✅ i = 5

s1[5] = 'n'
s2[5] = 't'

n → 13
t → 19

count[13]++ → (-1 + 1 = 0)
count[19]-- → (1 - 1 = 0)

🔥 Final Count Array

All values = 0 ✅

🎯 Final Conclusion

👉 Agar final me:

count[i] == 0 for all i

➡️ Strings Anagram hain ✅

👉 Agar kisi bhi index pe non-zero mila
➡️ Not Anagram ❌

💡 Simple Understanding Trick

Socho:

s1 → "deposit kar raha hai" (+1)

s2 → "withdraw kar raha hai" (-1)

👉 Agar balance 0 ho gaya → sab match ✔️
👉 Agar balance non-zero → mismatch ❌